

DOCUMENTACIÓN TÉCNICA COMPLETA DEL SISTEMA

Arquitectura, Docker, PostgreSQL, Backend REST y Frontend SPA

Autor: Mauro | Sistema de Gestión y Registro de Reclamos | Septiembre 2026

1. Introducción y Stack Tecnológico

Este proyecto sustituye una planilla compartida de Excel online por una solución web profesional, segura y concurrente. A continuación se detallan las tecnologías y programas utilizados en su construcción:

Motor de Base de Datos	PostgreSQL 16 (Alpine Linux) en contenedor Docker con volumen persistente.
Virtualización / Contenedores	Docker Desktop y Docker Compose para empaquetado y orquestación multi-servicio.
Entorno de Ejecución Backend	Node.js v22 LTS con arquitectura RESTful modular en capas.
Framework de Servidor Web	Express v5.x con middlewares de CORS, JSON, static files y enrutamiento.
Criptografía y Seguridad	Bcryptjs (hashing con 10 rondas de salt) y JSON Web Tokens (JWT) con caducidad.
Conexión a Base de Datos	pg (node-postgres) con Connection Pooling optimizado.
Procesamiento Multimedia	Multer para subida de fotos de perfil almacenadas en disco local.
Generador de Reportes	PDFKit para maquetación vectorial y descarga de reportes filtrados en PDF.
Frontend	Single Page Application (SPA) en HTML5 semántico, CSS3 corporativo y Vanilla JS modular.

2. Arquitectura de Red y Contenedorización con Docker

- Aislamiento de Puertos: En la máquina de desarrollo ya existía un PostgreSQL en el puerto estándar 5432. Para evitar conflictos, en docker-compose.yml se configuró el mapeo "5433:5432" (host:contenedor).
- Persistencia de Datos: Se definió el volumen "reclamos_pgdata" para que los datos nunca se borren al apagar el contenedor.
- Inicialización Automatizada: Se vinculó el archivo "database/init.sql" al directorio "/docker-entrypoint-initdb.d/". Al crearse el contenedor por primera vez, PostgreSQL ejecuta automáticamente todas las tablas, relaciones y catálogos iniciales.
- Producción: Se creó el Dockerfile con imagen liviana Node 22-alpine para empaquetar toda la aplicación.

3. Modelo de Datos Relacional (PostgreSQL)

La base de datos modela la jerarquía en cascada de las consultas de clientes: Tipo de Consulta -> Características -> Definición (Opcional) -> Finalización (Opcional). Consta de 8 tablas relacionales:

usuarios	id, nombre_completo, documento (UQ), usuario (UQ), password_hash, rol, activo, foto_perfil, created_at.
sucursales	id, nombre, activo, created_at (5 sucursales precargadas).
tipos_consulta	id, contenido, descripcion, activo (6 tipos: Técnicos, Facturación, Venta, Baja, Generales, Baja Parcial).
caracteristicas_consulta	id, tipo_consulta_id (FK), contenido, descripcion, activo (29 características vinculadas).
definiciones_consulta	id, caracteristica_id (FK), contenido, descripcion, activo (61 definiciones cargadas).
finalizaciones	id, definicion_id (FK), contenido, descripcion, activo (105 opciones en cascada).
reclamos	id, fecha, sucursal_id (FK), usuario_id (FK), numero_cliente, tipo_id, car_id, def_id, fin_id, timestamps.
auditoria_logs	id, fecha, accion, usuario_id, usuario_nombre, entidad, registro_id, datos_anteriores (JSONB), datos_nuevos.

4. Matriz de Seguridad y Permisos por Jerarquía (RBAC)

PESTAÑA / FUNCIONALIDAD	ADMIN	SUPERVISOR	ASESOR	ESPECTADOR
Gestión de Reclamos (Carga)	Completo	Completo	Completo	Sin acceso
Historial (Filtros y Consulta)	Completo	Completo	Completo	Solo lectura
Historial (Editar Reclamo)	Sí	Sí	Sí	No
Historial (Eliminar Reclamo)	Sí (Exclusivo)	No	No	No
Historial (Emitir Reporte PDF)	Sí	Sí	Sí	No
Registros (Auditoría Paginada x20)	Total (Exclusivo)	Sin acceso	Sin acceso	Sin acceso
Edición (Estructura y Sucursales)	Crear/Editar/Borrar	Crear/Editar	Sin acceso	Sin acceso
Perfil Propio (Clave y Foto)	Sí	Sí	Sí	Sí
Gestión Usuarios (En Perfil)	Total (Todos)	Solo Asesores	Sin acceso	Sin acceso
Activar / Suspender Cuentas	Sí (Exclusivo)	No	No	No

5. Pestañas y Experiencia del Frontend (SPA)

- Gestión de Reclamos: Selects dinámicos en cascada, auto-asignación de fecha del sistema y asesor en sesión. Ancho completo.
- Historial: Filtros combinados por fecha exacta, rango, sucursal, tipo, asesor y cliente. Tabla con acciones y botón PDF.
- Registros: Paginación estricta de 20 logs por página. Muestra fecha, acción, responsable y snapshot exacto del dato anterior.
- Edición: Vista de árbol jerárquico para crear, modificar o eliminar nodos de consulta y catálogo de sucursales.
- Perfil: Modificación de clave validando la actual, subida de foto de perfil y panel de administración de cuentas para Admin.

6. Cronograma de Desarrollo Realizado (Paso a Paso)

1. Diagnóstico de puerto y configuración de docker-compose.yml con puerto 5433:5432 y volumen local persistente.
2. Creación del script database/init.sql con DDL relacional y carga completa del árbol jerárquico extraído del Excel.
3. Despliegue de PostgreSQL con "docker compose up -d" y verificación de conectividad y recuentos mediante psql.
4. Implementación del Backend en Express con autenticación JWT, hashing bcrypt y middleware interceptor de auditoría.
5. Desarrollo de controladores y rutas para reclamos, catálogos, usuarios, auditoría paginada y reportes PDF con pdfkit.
6. Construcción del frontend SPA modular: api.js, auth.js, reclamos.js, historial.js, registros.js, edicion.js, perfil.js y app.js.
7. Ejecución de suite de 10 pruebas automatizadas end-to-end con Node.js verificando todas las operaciones CRUD y PDF.
8. Generación del Dockerfile para empaquetado del frontend/backend listo para despliegue productivo.
9. Generación de las guías formales en PDF del Plan de Implementación y de Migración de Docker entre computadoras.
10. Ajustes requeridos: retiro de rol en header, formulario en ancho completo, sección de usuarios en perfil para Admin.
11. Retiro de la opción "Legajo" en toda la base de datos, backend y frontend.
12. Implementación de suspensión/activación inmediata con mensaje estricto "Usuario suspendido" en caso de bloqueo de acceso.
13. Ajuste de jerarquía flexbox y scroll vertical en los modales modal-crear-usuario y modal-editar-usuario.
14. Limpieza total de comentarios en el código y adición de firma de autoría de Mauro al final de index.html.
15. Aplicación del rediseño estético profesional y corporativo de alta eficiencia.

7. Comandos Esenciales para Operación y Estudio

docker compose up -d	Inicia los contenedores en segundo plano.
docker ps	Muestra los contenedores en ejecución y sus puertos mapeados.
docker logs -f reclamos_postgres	Muestra los registros en vivo del motor de base de datos.
docker exec -it reclamos_postgres psql -U postgres -d reclamos_db	Abre la consola SQL dentro del contenedor.
docker exec -t reclamos_postgres pg_dump -U postgres -d reclamos_db > backup.sql	Exporta una copia de seguridad completa.
Get-Content backup.sql docker exec -i reclamos_postgres psql -U postgres -d reclamos_db	Restaura una copia en PowerShell.
node src/server.js	Inicia el servidor Node.js de la aplicación.